IMPERIAL

agent-spec-kit

Scenario-Driven Evaluation and Testing of Tool-Using LLM Agents

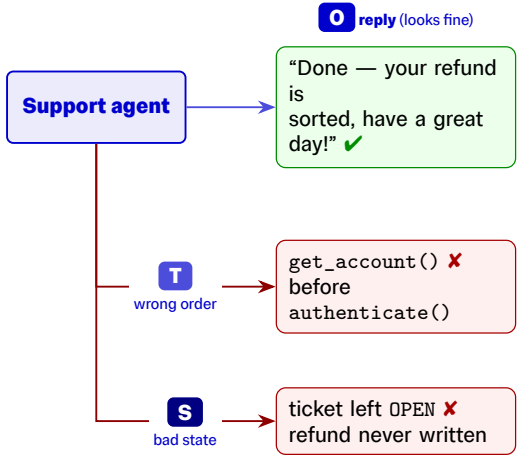
Ruthvik Konduru

MEng Individual Project --- Department of Computing

Tool-using agents act on the world

- LLMs are no longer just text generators — as **agents** they plan, call tools, and **change external state**.
- Correctness can therefore **not** be judged from the final reply alone.
- A support agent can sound perfectly polite while it:
  - skips **authentication** and leaks account data,
  - calls the **wrong tool** or in the wrong order,
  - leaves the **database** in an invalid state.

The plausible answer is only one part of being correct.



The reply passes; the actions and state silently do not.

Correctness lives on three surfaces

Output is necessary but far from sufficient

Output

what the agent says

Trace

which tools it called

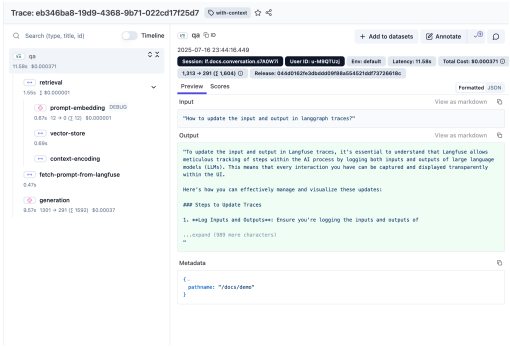
State

the state it left behind

A failure on any surface can be hidden by a fluent reply — and may even be masked by recovery on a later turn.

Existing tools only cover part of the loop

- **Output graders / LLM-as-judge** (HELM, G-Eval, MT-Bench): great for comparing models on isolated final answers — but blind to tool and state failures.
- **Agent benchmarks** (τ-bench, AppWorld, WebArena): trace/state aware, but fixed leaderboards, not reusable infrastructure for your own agent.
- **Trace viewers** (LangSmith, Langfuse): show what happened — the developer still has to infer what should have happened.



Trace viewers show the episode; they do not encode the expected behaviour.

The gap

No single, reusable way to assert over **output + trace + state** in one multi-turn scenario — with **precise diagnostics** when a check fails.

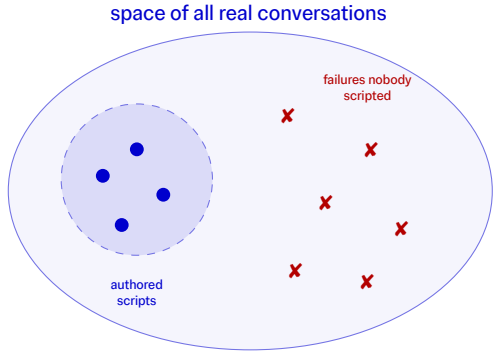
...and you can't script every conversation

Authored tests only cover what you thought to write

- Hand-written scenarios capture the policies you already know — the cases you thought of.
- But agents are **stochastic** and real users are **messy**: they rephrase, omit details, push back, change intent mid-conversation.
- The failures that bite are the dialogue paths **nobody scripted**.

Beyond the cases you authored

The framework should **explore** conversations automatically — simulated users and transcript fuzzing — and **keep** the real failures it finds as permanent regression tests.



Scripts cover a small known region — bugs hide in the rest.

Research question

> Can the evaluation of **stateful, tool-using LLM agents** be expressed in a **software-testing style**:
>
> readable scenarios that combine assertions over **replies**, **tool trajectories**, and **application state**, while keeping enough structured evidence to **diagnose** failures precisely?

Answer: agent-spec-kit — a scenario-driven testing framework for LLM agents, evaluated on a 50-task benchmark.

IMPERIAL

Part I --- The Library

agent-spec-kit: behaviour as executable scenarios

The core idea: a scenario is a conversation script

Output, tool, and state checks live together, woven in after each turn

```python
@scenario(agent_fixture="agent", user_fixture="user")
async def test_billing_credit(s, store):
    s.user_message("I was double charged in October")
    # O: the reply must insist on verifying first
    s.assert_output(m.contains("verify"))
    # T: the agent must authenticate before account tools
    s.assert_tool_calls([
        m.tool_call("authenticate_customer"),
        m.tool_call("get_billing", args={"id": "CUST-045"}),
    ])
    # S: a postcondition on the database state
    s.assert_that(assert_credit_policy)
```

- **O** `assert_output` — the reply
- **T** `assert_tool_calls` / `forbid_tool_calls` — the trace
- **S** `assert_that` — environment / DB state

One file reads like a dialogue with the contract inlined — no separate datasets, graders, and runner config.

One composable matcher language

Start simple: matching the reply

- Literals, `dicts` and `lists` are coerced into matchers automatically.
- Combine them: `m.one_of` (any), `m.all_of` (all), `m.not_`, `m.contains`, `m.regex`.
- Selective **LLM rubric** only where text needs judgement — pass/fail criteria, not opaque scores.

```
# deterministic: offer a refund OR a credit
s.assert_output(
  m.one_of(
    m.contains("refund"),
    m.contains("account credit"),
  )
)
```

```
# nuance? hand that one check to an LLM judge
s.assert_output(m.llm_criteria(
  criteria=["acknowledges the double charge",
            "does not promise a refund yet"],
  threshold=2,
))
```

Matching the tool trace

Order and extras are explicit, not implied

- `assert_tool_calls` matches the normalised tool trace.
- `ordered=True` pins the sequence; `ordered=False` for parallel siblings.
- `allow_extras=True` lets other calls interleave; `forbid_tool_calls` asserts a tool is never used.

```
s.assert_tool_calls(
  [
    m.tool_call("authenticate_customer"),
    m.tool_call("get_billing",
                args={"id": "CUST-045"}),
  ],
  ordered=True, # must appear in order
  allow_extras=True, # others may interleave
)
```

...and the matchers compose

Structured objects and conditional rules

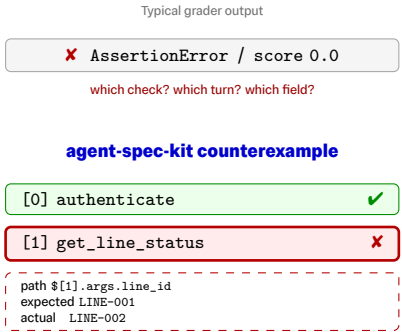
- `m.object` constrains structured args or results — only the fields you name.
- `m.list` matches sequences `ordered` or `unordered`.
- **Conditional rules**: `require`/`forbid` a field only when another field holds.

```
m.tool_call("open_incident", args=m.object(
  {
    "id": m.regex(r"INC-\d{4}-\d+"),
    "severity": m.one_of("high", "medium"),
    "components": m.list(["edge", "core"],
                         mode="unordered"),
  },
  rules=[ # conditional
    m.require("pager_group")
     .when(m.field("severity") == "high"),
    m.forbid("pager_group")
     .when(m.field("severity") == "medium"),
  ],
))
```

Precise diagnostics: structured counterexamples

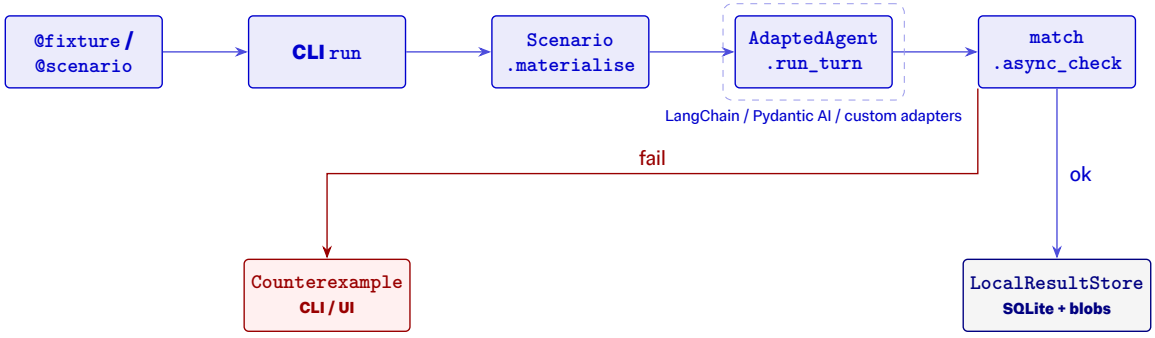
When a check fails, you learn which expectation broke, where

- Every failed assertion produces a **Counterexample**:
  - headline + human location (“assert_tool_calls after turn #2”)
  - a **JSON-pointer path** to the mismatch, e.g. `$[1].args.line_id`
  - expected vs. actual, plus a bounded event trace
- Identical view in the **terminal**, the **SQLite** store, and the **web UI**.
- Turns “it failed somehow” into “this check broke here”.



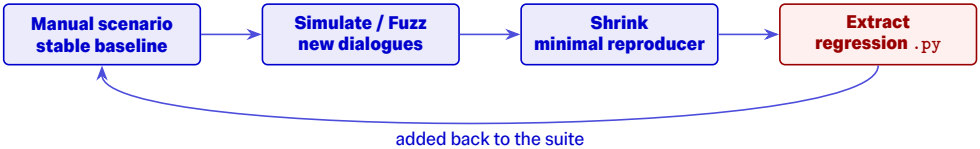
A scalar tells you that it failed; a counterexample tells you where.

Framework-neutral execution model



- Adapters **normalise traces** (LangGraph, Pydantic AI) into one event model — oracles never depend on a framework's message format.

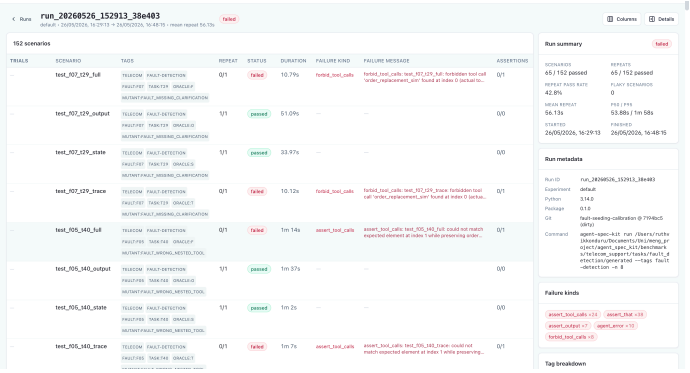
From authored tests to automatic discovery



- **Repeats** for stochastic agents.
- **User simulation**: persona-driven multi-turn dialogues.
- **Mutation fuzzing**: perturb valid transcripts.
- **Shrinking**: delta-debug to a minimal failing dialogue.
- **Extraction**: pin a discovered failure as a permanent scenario.
- All under the same oracle vocabulary.

Persistent runs, CLI, and a web results browser

- Every run stored locally (SQLite + gzip blobs) with git metadata.
- CLI: `run`, `runs`, `show`, `regressions`, `ui`.
- Read-only **web UI**: runs list, run detail, trace drawer, fuzz trials.
- **Compare** named experiments side-by-side to track regressions.



Run detail page: per-scenario, per-repeat results with a trace drawer.

Live Demo

Writing a scenario, watching it fail precisely, discovering a bug, and pinning it as a regression.

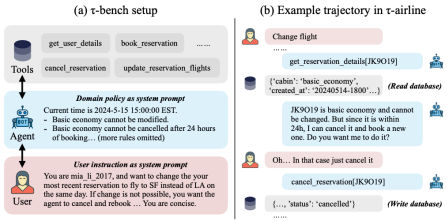
(≈ 8--10 minutes --- see speaker plan for the script)

Part II --- Evaluation

Does multi-surface, scenario-driven testing actually pay off?

TelcoSupportBench: the evaluation testbed

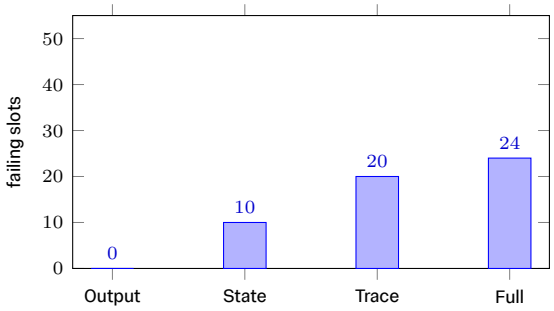
- A synthetic **mobile-support** domain built for this project.
- **50 tasks** (T01–T50) against a reference **LangGraph** agent: a coordinator + diagnostics + billing specialists.
- Each task has **four oracle lanes**: **O** output, **T** trace, **S** state, and **F** = full (all three).
- **Design principle:** tools stay permissive; **policy is asserted in the oracles**, not hidden in the simulator.
- Compared against **7** other evaluation approaches.



Inspiration: τ-bench-style dual-control support setting. **[Replace with TelcoSupportBench architecture figure if available]**

Finding 1 --- output-only grading is not enough

Reference agent across 200 oracle slots (50 tasks × 4 lanes)



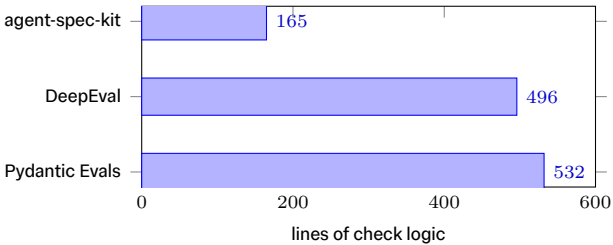
- **All 50 output lanes passed** — yet **54 of 200** slots failed.
- Every one of those failures lived in the **trace**, **state**, or **full** lanes.
- The reply “sounded correct” while a tool ran in the wrong order, an argument was wrong, or a store update never happened.

Takeaway

Final-answer evaluation alone would have reported **100%** pass.

Finding 2 --- more expressive, less code

Twelve canonical check patterns, ported across frameworks



- **165 lines** vs. 496 (DeepEval) and 532 (Pydantic Evals).
- **Highest clarity grade on every one** of the twelve specimens.
- On failures, it gives an exact **path witness** where alternatives return a scalar, broad prose, or a bare `AssertionError`.

Declarative DSL — equivalent in policy to the hand-written ports, but far easier to read and debug.

Finding 3 --- full oracles catch faults narrow lanes miss

Injected-fault detection (F01--F10) on previously-passing slots

- Ten fault families injected into the agent; measure which lane detects each.
- **Output** detection is frequently **0%** for tool-ordering and state-mutation faults.
- **State** and **Full** lanes reach **75–100%** on those same faults.
- The **combined oracle** is stronger than any single surface alone.

Fault	O	S	T	F
F02 (state)	0%	**100%**	67%	**100%**
F03 (state)	0%	**100%**	0%	**100%**
F05 (order)	0%	0%	**100%**	**100%**
F07 (trace)	50%	50%	**100%**	**100%**
F08 (trace)	0%	0%	**75%**	50%
F10 (full)	25%	0%	33%	**100%**

Detection rate = detected / eligible. Selected rows; full matrix F01–F10 in the report.

Finding 4 --- generative testing finds bugs nobody scripted

14 tasks, the same F-oracle throughout

Method	Convs	Tool paths	Failure sigs
Manual scripts	14	10	3
User simulation	70	**58**	**24**
Mutation fuzz	70	36	18

- Simulation & fuzzing reach **far more failure signatures** than the hand-written scripts — at little extra authoring cost.
- Discovered failures are **shrunk** and **extracted** into permanent regressions.

Threats to validity

- **Construct:** clarity & diagnostic grades used **LLM judges**, not a human developer study.
- **External:** a **single** reference agent and **one** telecom domain — design choices may favour the framework.
- **Rerun:** extraction produced a valid file for every eligible failure, but **fewer than half** reproduced the exact signature on immediate rerun — stochastic ordering/wording drifts.
- Results are **promising** but describe this setup, not agents in general.

Conclusion

- The right unit of evaluation is the **behavioural episode**, not the final reply — a polite answer can hide a skipped auth step, a mis-ordered tool, or a corrupted store.
- agent-spec-kit unifies **output + trace + state** oracles in one readable, executable, framework-neutral scenario.
- **Diagnostics are part of evaluation quality**: assertion-local counterexamples make failures fixable, not just countable.
- A single workflow spans **discovery** $\rightarrow$ **regression**: simulate, fuzz, shrink, extract.
- Evidence: hidden trace/state failures exposed, **3**$\times$ **less** check code with the highest clarity, and generative testing surfacing real bugs — e.g. a pre-auth privacy leak.

Future work

- More **domains and agent frameworks** — test whether the scenario model transfers beyond TelcoSupportBench.
- **Human studies** of authoring clarity and diagnostic usefulness, to replace LLM-judge grading.
- **Signature-stable replay** after extraction, and more robust matching of stochastic trajectories.
- **Larger, team-based suites** to see how the workflow scales in everyday development.

IMPERIAL

Thank you.
Questions?

agent-spec-kit
June 2026